

Task and Session State Store (MongoDB)

Task and Session State Store (MongoDB)

Overview

This document defines the **MongoDB state store** used by the Gateway and Agent Harness. It provides persistent storage for:

- Task lifecycle state
- Session lifecycle state
- Human-in-the-loop interaction data
- Agent snapshots
- Workflow execution metadata

All models are written using **Mongoose** and designed for:

- Fast lookups
- Atomic updates
- Polyglot agent compatibility
- Human-in-the-loop pause/resume workflows
- Redis event integration

Folder Structure

```
/db
index.ts
models/
  Task.ts
  Session.ts
  HumanInteraction.ts
  AgentSnapshot.ts
  WorkflowState.ts
```

File: index.ts

```
import mongoose from "mongoose";

export async function connectDB() {
  const uri = process.env.MONGO_URI || "mongodb://localhost:27017/agent_system";

  await mongoose.connect(uri, {
    autoIndex: true
  });

  console.log("MongoDB connected");
}
```

 File: models/Task.ts

```

import { Schema, model } from "mongoose";

const HumanInteractionSchema = new Schema({
  promptMessage: String,
  options: [String],
  selectedChoice: String,
  updatedAt: Date
});

const AgentSnapshotSchema = new Schema({
  data: Schema.Types.Mixed,
  updatedAt: Date
});

const TaskSchema = new Schema(
{
  agentId: { type: String, required: true },
  input: { type: Schema.Types.Mixed, required: true },

  status: {
    type: String,
    enum: [
      "PENDING",
      "RUNNING",
      "WAITING_FOR_HUMAN",
      "QUEUED",
      "COMPLETED",
      "FAILED"
    ],
    default: "PENDING"
  },

  humanInteraction: HumanInteractionSchema,
  agentInternalSnapshot: AgentSnapshotSchema,

  createdAt: { type: Date, default: Date.now },
  updatedAt: { type: Date, default: Date.now }
},
{ timestamps: true }
);

TaskSchema.index({ status: 1 });
TaskSchema.index({ agentId: 1 });

export const TaskModel = model("Task", TaskSchema);

```

File: models/Session.ts

```
import { Schema, model } from "mongoose";

const SessionSchema = new Schema(
  {
    sessionId: { type: String, required: true },
    agentId: { type: String, required: true },

    state: {
      type: String,
      enum: ["idle", "running", "waiting", "completed", "failed"],
      default: "idle"
    },

    lastEventAt: { type: Date, default: Date.now },

    createdAt: { type: Date, default: Date.now }
  },
  { timestamps: true }
);

SessionSchema.index({ sessionId: 1 });
SessionSchema.index({ agentId: 1 });

export const SessionModel = model("Session", SessionSchema);
```

File: models/HumanInteraction.ts

```
import { Schema, model } from "mongoose";

const HumanInteractionSchema = new Schema(
  {
    taskId: { type: String, required: true },

    promptMessage: { type: String, required: true },
    options: { type: [String], required: true },
    selectedChoice: { type: String },

    createdAt: { type: Date, default: Date.now },
    updatedAt: { type: Date, default: Date.now }
  },
  { timestamps: true }
);

HumanInteractionSchema.index({ taskId: 1 });

export const HumanInteractionModel = model("HumanInteraction",
HumanInteractionSchema);
```

File: models/AgentSnapshot.ts

```
import { Schema, model } from "mongoose";

const AgentSnapshotSchema = new Schema(
  {
    taskId: { type: String, required: true },
    data: { type: Schema.Types.Mixed, required: true },
    updatedAt: { type: Date, default: Date.now }
  },
  { timestamps: true }
);

AgentSnapshotSchema.index({ taskId: 1 });

export const AgentSnapshotModel = model("AgentSnapshot", AgentSnapshotSchema);
```

File: models/WorkflowState.ts

```
import { Schema, model } from "mongoose";

const WorkflowStateSchema = new Schema(
  {
    workflowId: { type: String, required: true },
    sessionId: { type: String, required: true },

    nodeStates: {
      type: Map,
      of: Schema.Types.Mixed
    },

    createdAt: { type: Date, default: Date.now },
    updatedAt: { type: Date, default: Date.now }
  },
  { timestamps: true }
);

WorkflowStateSchema.index({ workflowId: 1 });
WorkflowStateSchema.index({ sessionId: 1 });

export const WorkflowStateModel = model("WorkflowState", WorkflowStateSchema);
```

Integration Notes

Gateway Responsibilities

- Create tasks and sessions
- Update task status (WAITING_FOR_HUMAN, QUEUED, etc.)
- Store human interaction choices
- Store workflow state

Agent Harness Responsibilities

- Save agent snapshots before pausing
- Load snapshots when resuming
- Update task status (RUNNING, COMPLETED, FAILED)

Frontend Responsibilities

- Display session state
 - Display human interaction prompts
 - Submit approval choices
-

Summary

This MongoDB state store provides:

- Persistent task and session lifecycle tracking
- Human-in-the-loop workflow support
- Agent snapshot storage
- Workflow execution metadata
- Indexed schemas for fast lookups

It is fully aligned with your Gateway, Agent Harness, Redis event schema, and frontend SSE architecture.